



Assignment 06: Working with a Typological Database

handed out: 4 June, 14:00
to be submitted by: 18 June, 12:00

In this assignment, we are going to work with typological data from the Grambank¹ database. The data are distributed in CLDF format, and need to be downloaded here <https://zenodo.org/records/7844558> as a ZIP archive which you need to unpack. We are only going to work with the files `languages.csv` (language information), `parameters.csv` (information about typological variables), `values.csv` (where the values of the parameters are coded), and `codes.csv` (where the code IDs used in `values.csv` are defined).

Task 1: Merging the CSV Files into a Single Dataframe

- Load the contents of the four files into Pandas DataFrame objects (suggested names correspond to file names without the CSV ending: `languages`, `parameters`, `values`, `codes`). Our goal is to merge the information about the typological parameters that will be relevant to our study into the `values` dataframe.
- Reduce the contents of your `values` dataframe to only those lines which contain values for the parameters GB020, GB021, GB022 (these are the only three features we will be analysing!), and remove the columns ID, Comment, Source, Source_comment, and Coders which were loaded from `values.csv`
- Merge the `Name` and `Macroarea` columns from your `languages` dataframe to your `values` dataframe using the language ID as keys.
- Merge the value descriptions (`Description`) from the `codes` dataframe into your `values` dataframe, using the value code as the left key and ID as the right key.

After performing this task, the last three lines in your `values` dataframe should look roughly like this:

	Language_ID	Parameter_ID	Value	Code_ID	ID_x	Name	Macroarea	ID_y	Description
7146	zuni1245	GB020	1	GB020-1	zuni1245	Zuni	North America	GB020-1	present
7147	zuni1245	GB022	0	GB022-0	zuni1245	Zuni	North America	GB022-0	absent
7148	zuni1245	GB023	0	GB023-0	zuni1245	Zuni	North America	GB023-0	absent

Task 2: Getting the values Dataframe into Shape

We are now going to reshape `values` into a long format with just four columns, as the basis for further reshaping operations.

- Remove the columns that are not relevant any more after the mergers because they contain duplicate or spurious information, and rename the remaining columns in such a way that language names occur in a `Language` column, parameter IDs in a `Feature` column, and the value descriptions in a `Value` column.
- Reorder the columns in the following order: `Language`, `Macroarea`, `Feature`, `Value`, and sort the result by macroarea, and within each macroarea by language name. Save the result again as `values`.
- Remove lines with incomplete data, and replace the feature names "GB020", "GB022", "GB023" by the more mnemonic "defArt" (existence of a definite article), "prenom" (prenominal position, i.e. before the noun), and "postnom" (postnominal position, i.e. after the noun).

After performing this task, the last three lines in your `values` dataframe should look roughly like this:

	Language	Macroarea	Feature	Value
6914	Yámana	South America	defArt	absent
6915	Yámana	South America	prenom	absent
6916	Yámana	South America	postnom	present

¹<https://grambank.clld.org/>

Task 3: Hierarchical Indexing

Starting with **values** in the long format, it is time to play around with hierarchical indexing.

- Modify **values** by creating a hierarchical row index with the language name as the inner level, and the macroarea as the outer level.
- What is the global distribution of presence vs. absence of definite articles? Use the partial indexing of **values** in order to compare your global result to the ones on languages of Eurasia and Africa.
- Reindex the values dataframe with a different hierarchical row index with the feature as the outer level and the macroarea as the inner level. Store the result under the name **reindexed**. Sort the result in such a way that values of the same feature appear in adjacent rows.
- You decide that this reindexing did not lead to the desired result. Change the index of **reindexed** in such a way that the macroarea becomes the outer level, and the feature gets moved to the inner level. Use cross-section on the resulting index in order to extract all languages with prenominal article position.

Task 4: Pivoting and Melting

In this task, we are going to continue working with the version of the **values** table which was the result of Task 3 a), i.e. with a two-level hierarchical index with macroarea as the outer level and language as the inner level.

- Decide whether to use pivoting or melting in order to cause three feature for every language to be arranged in a row, with the three feature names occurring as column names. Store the resulting dataframe under the name **languages_rows**.
- The result will have an unnecessary outer level in the column index. Remove it by calling the **droplevel()** method (we did not cover it during the session, read up in the Pandas documentation how the method works in case it is not immediately clear).
- Use the reshaped dataframe to answer the question which languages feature definite articles in both postnominal and prenominal position.
- Which three method calls can we chain together to transform this version of **languages_rows** back into a format that is as similar as possible to the original version of **values** which we had at the end of Task 2?

Task 5: Concatenation

- We are now going to consider how the pivot operation from Task 4 could be performed by more elementary concat operations. For this purpose, extract from 'values' three dataframes which each contain only the values of one of the three variables, and store them under the names **def_art_data**, **prenom_data**, and **postnom_data**.
- Concatenate the three dataframes along the column axis, making sure that the data for the three features remain identifiable by handing over the feature names as keys. Use the option which allows you to limit the output to complete rows.
- Explore the alternative option of concatenating the three dataframes along the row axis, again making sure the features remain identifiable by an additional outermost layer of the hierarchical row index. Extract the single column of the resulting dataframe in order to have the entire data in a Series object with a three-level hierarchical index.